# La gestione dei processi

**Algoritmo**

Procedimento logico (sequenza di operazioni) che deve essere seguito per arrivare alla soluzione di un problema.

**Programma**

Descrizione dell'algoritmo tramite un opportuno linguaggio di programmazione, che rende possibile l'esecuzione dell'algoritmo da parte di un elaboratore.

Un programma, quindi, possiamo vederlo come una entità statica, in quanto è semplicemente un insieme di istruzioni memorizzate su un file presente in una memoria di massa, in attesa di essere caricate in RAM ed eseguite.

Un processo, come già abbiamo detto in precedenza, è un programma in esecuzione; per meglio dire, è una **istanza** di un programma in esecuzione, in quanto un programma potrebbe avere associati più processi. Infatti, se apriamo tre documenti con lo stesso editor di testo (ad esempio Word), a livello di sistema avremo che ogni finestra aperta (ogni documento) corrisponderà ad un processo in esecuzione dello stesso programma.

Un processo, detto anche **task**, è l'unità funzionale di un qualsiasi Sistema Operativo; è controllato da un programma e necessita di un processore per essere eseguito.

Qualsiasi processo ha bisogno di **risorse** durante la sua esecuzione, ovvero di un qualche elemento necessario all'avanzamento del processo stesso. Una risorsa può essere la CPU, un certo spazio di memoria RAM, una periferica, un file, …

Alcune risorse possono essere utilizzate da più processi contemporaneamente (ad esempio un file aperto in lettura), mentre altre risorse sono dette **seriali**, in quanto possono essere utilizzate da un solo processo alla volta. Prendiamo come esempio la stampante: sarebbe "disastroso" se più processi alla volta potessero utilizzarla contemporaneamente.

Il Sistema Operativo ha il delicatissimo e fondamentale compito di assegnare, secondo un qualche criterio, le risorse ai processi che le richiedono, evitando che

alcuni processi non riescano mai ad accedere alle risorse di cui necessitano (**starvation**), mentre altri le mantengano per troppo tempo.

Una caratteristica fondamentale che si ha quando la risorsa in questione è la CPU, è la cosiddetta **preemption**, ovvero la possibilità del S.O. di forzare il rilascio della CPU da parte del processo, per consentire ad altri processi di essere mandati in esecuzione.

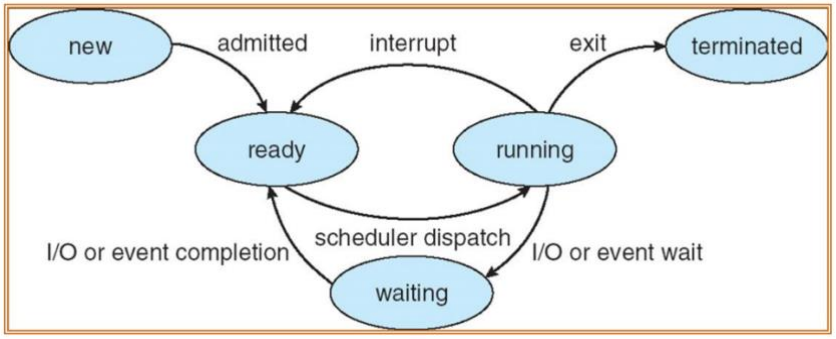
Chiaramente all'interno del sistema possono essere presenti processi più importanti di altri, ovvero processi con una **priorità** maggiore; questa priorità può essere assegnata in modi diversi, a seconda del tipo di sistema. Per esempio, possiamo avere casi in cui la priorità è tanto più alta quanto minore è la durata del processo; tipicamente i processi di sistema hanno priorità maggiore dei processi utente.

Processi con più alta priorità sono privilegiati nell'utilizzo delle risorse; questo significa che se un processo ha bisogno di una risorsa occupata da un processo con priorità inferiore, questa viene rilasciata per consentirne l'utilizzo da parte del processo a priorità maggiore.

Pertanto, avremo bisogno di un particolare processo di sistema, detto **scheduler**, che, per ogni tipo di risorsa, decida, secondo una qualche politica, a quale processo assegnare la risorsa stessa.

# Stati di un processo

Un processo può trovarsi in diversi stati di avanzamento.




Analizziamo gli stati visualizzati in figura:

- **new**: il processo è appena stato creato, è memorizzato su disco (o comunque su un supporto di memorizzazione di massa) ed è in attesa di essere caricato in memoria centrale. Tipicamente, il S.O. deve gestire una lista di più processi che si trovano in questo stato ed un particolare processo, detto **Long Term Scheduler** (schedulatore a lungo termine)

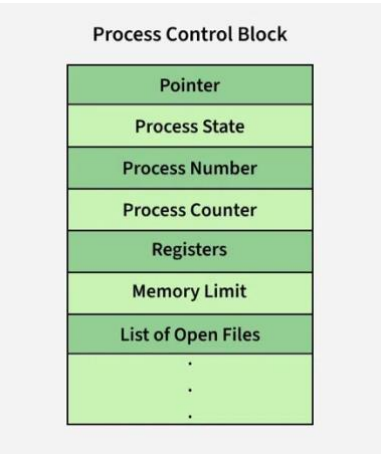
deve decidere quali di questi processi caricare in RAM e quali lasciare ancora su disco;
- **ready**: il processo è pronto, ed è in attesa che lo scheduler lo selezioni per essere mandato in esecuzione, ovvero che gli venga assegnata la CPU (o una delle CPU). In questo caso entra in gioco un altro schedulatore, detto **Short Term Scheduler** (schedulatore a breve termine), che decide proprio quale processo può essere mandato in esecuzione, secondo una qualche politica;
- **running**: il processo è in esecuzione. In ogni istante, ci possono essere tanti processi in esecuzione quanti sono i processori presenti sulla macchina. Per garantire un certo tempo di esecuzione a tutti i processi in attesa (nello stato ready), di solito un processo ha a disposizione un certo quanto di tempo, detto **time slice**, trascorso il quale gli viene revocata la CPU se la sua esecuzione non dovesse essere ancora terminata;
- **waiting**: il processo è in attesa di un qualche evento, ad esempio che termini una operazione di I/O. Anche in questo caso il S.O. ha a che fare con una lista di processi bloccati. Le operazioni di I/O vengono eseguite da processori dedicati per l'I/O, che lavorano in parallelo alla CPU;
- **terminated**: il programma ha terminato la sua esecuzione e, quindi, lascia il sistema, liberando le risorse che stava utilizzando.

# PCB (Process Control Block)

Ad ogni processo viene associata una tabella che racchiude alcune informazioni riguardanti il processo stesso: il PCB (Process Control Block), che viene inizializzato durante la creazione del processo e mantenuto in memoria (e aggiornato di volta in volta) in una apposita area accessibile solamente dal S.O., per tutta la durata del processo.

In questa area di memoria, quindi, troveremo una sorta di tabella contenente tu tti i PCB associati ad ogni processo presente nel sistema.

Tra le informazioni contenute nel PCB, che possono cambiare in base al S.O., abbiamo:




- lo stato del processo (pronto, in esecuzione, bloccato, …);
- il PID (Process ID), ovvero un identificativo del processo;
- l'identificatore (PID) del processo padre;
- un puntatore alla lista dei processi figli;
- i limiti della memoria centrale utilizzata;
- un puntatore alla lista delle risorse che il processo sta utilizzando;

- uno spazio contenente una copia del contenuto dei registri della CPU;
- la priorità del processo, che influisce sul suo tempo di attesa;
- un puntatore alla lista dei **threads** del processo (se supportati).

Il PCB è fondamentale per consentire la commutazione della CPU tra diversi processi; infatti, quando la CPU deve essere assegnata ad un altro processo che deve essere mandato in esecuzione, prima di effettuare lo scambio è necessario aggiornare e salvare tutte le informazioni del PCB del processo che è attualmente in esecuzione.

Questo serve a garantire che il processo in questione, quando tornerà in esecuzione, possa riprendere da dove era stato interrotto, con tutti i dati che gli servono (occorre sapere i valori dei registri da lui utilizzati, quali risorse stava utilizzando, …).

# Scheduling dei processi

Abbiamo visto come esistono due tipi di scheduling dei processi:

- lo **scheduling a lungo termine**, che consiste nell'andare a decidere quali dei processi presenti su disco devono essere caricati in memoria RAM. Questo tipo di scheduling va a regolare il **grado di multiprogrammazione**, ovvero il numero di processi attivi nel sistema;
- lo **scheduling a breve termine**, che serve per decidere quale processo, tra quelli pronti per essere eseguiti (presenti in memoria RAM), mandare in esecuzione. Il processo che esegue questo compito prende anche il nome di **dispatcher**.

Il secondo tipo di scheduling avviene molto più frequentemente del primo.

**Context switch**

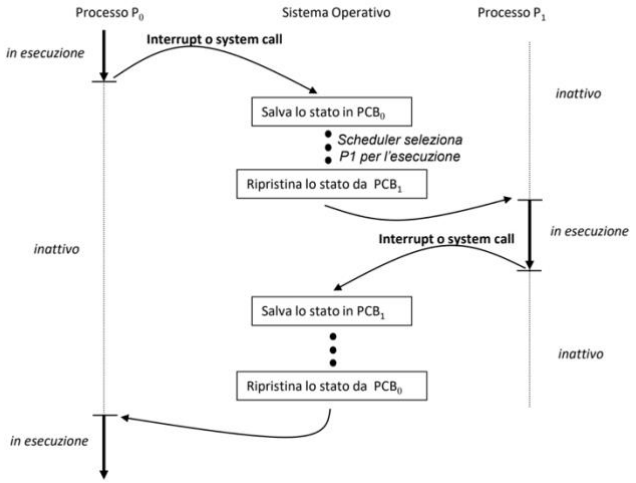
L'insieme di operazioni con le quali la CPU salva lo stato di un processo attualmente in esecuzione e carica quello di un altro processo.

Il context switch è fondamentale per il multitasking, ovvero avere l'illusione che più processi siano in esecuzione contemporaneamente (su un unico processore).

Le principali operazioni che vengono eseguite durante il cambio di contesto sono le seguenti:

1. viene aggiornato il PCB del processo attualmente in esecuzione, salvando il contenuto dei registri della CPU, che verrà ripristinato quando questa verrà riassegnata di nuovo al processo, in modo che l'esecuzione possa riprendere come se non fosse stata mai interrotta;

2. viene scelto il processo da mandare in esecuzione attraverso un algoritmo di scheduling;
3. viene caricato il PCB del processo selezionato dal dispatcher ed i valori dei registri in esso salvati vengono copiati nei registri della CPU;
4. il nuovo processo viene mandato in esecuzione.




Per entrambi i tipi di scheduling esistono diversi algoritmi, ciascuno con vantaggi e svantaggi.

## FCFS (First Come First Served)

Nel caso di scheduling a lungo termine, l'algoritmo va a caricare in memoria RAM i processi secondo l'ordine di arrivo. Ricordiamo che, una volta che un processo viene ammesso in memoria centrale, viene memorizzato in una coda di processi pronti (nello stato ready), ovvero quei processi che sono caricati in memoria RAM e stanno aspettando che lo scheduler assegni loro la CPU.

Se questo algoritmo viene utilizzato per lo scheduling a breve termine, la coda dei processi pronti verrà gestita in modalità FIFO (First In First Out), ovvero i processi verranno "serviti" nell'ordine con cui arrivano/vengono creati.

È un algoritmo molto semplice da implementare e che evita starvation, in quanto tutti i processi vengono eseguiti; tuttavia, il principale problema di questo algoritmo è che non considera la durata o la priorità di un processo e, quindi, si rischia che processi lunghi che si presentano per primi rallentino processi anche molto brevi, che però si presentano dopo.

## SJF (Shortest Job First)

Anche questo algoritmo è molto semplice da intuire: viene "accontentato" prima il processo con durata inferiore. Quindi i processi che durano di meno verranno caricati prima in RAM (scheduling a lungo termine) e avranno la precedenza anche per quanto riguarda l'assegnazione della CPU (scheduling a breve termine).

Questo algoritmo migliora l'efficienza generale del sistema, andando a ridurre il tempo medio di attesa dei processi, ovvero il tempo che intercorre da quando il processo è pronto per essere eseguito (è caricato in memoria RAM) a quando gli viene effettivamente assegnata la CPU.

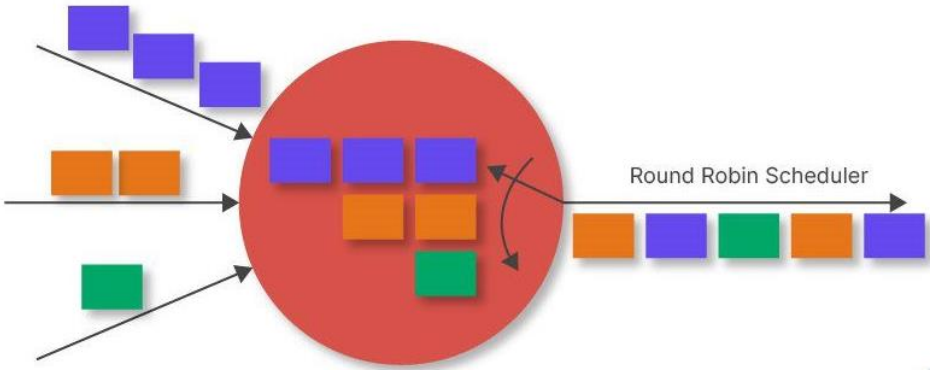
Tuttavia, spesso risulta difficile stimare correttamente la durata effettiva di un processo, soprattutto se questo richiede l'esecuzione di molte operazioni di I/O. Inoltre, con questo algoritmo si rischia di avere starvation: processi molto lunghi possono rimanere indefinitamente in attesa perché numerosi processi molto più brevi continuano ad avere la precedenza su di loro.

## RR (Round Robin)

È un algoritmo preemptive per lo scheduling a breve termine, che consiste nel gestire i processi tramite una coda gestita in maniera FIFO. Ad ogni processo viene assegnato un certo time slice per utilizzare la CPU; se al termine di questo tempo il processo non ha ancora terminato la sua esecuzione, gli viene comunque revocata la CPU, viene inserito in fondo alla coda e la CPU viene assegnata al processo che lo seguiva.

Questo algoritmo è molto semplice da implementare, ed ha il pregio di essere estremamente equo, in quanto viene garantito a tutti i processi lo stesso time slice di esecuzione; inoltre, non presenta starvation, perché un processo che non termina durante il time slice viene inserito in fondo alla coda, che ricordiamo essere gestita in modalità FIFO, garantendo così che tutti i processi riescano ad ottenere la CPU.

La sfida principale sta nel dimensionare correttamente la durata del quanto di tempo assegnato ai processi: se il time slice è troppo lungo (più lungo della durata massima prevista per un processo), l'algoritmo degenera e diventa un semplice FCFS, in quanto i processi vengono eseguiti estraendoli in ordine dalla coda dei processi pronti, mentre se è troppo corto, le prestazioni calano drasticamente, in quanto si avrebbe troppo overhead causato da continui context switch.




## Scheduling con priorità

Abbiamo visto come l'algoritmo Round Robin sia molto equo nello scheduling dei processi; tuttavia, non tiene conto del fatto che alcuni processi possono essere più importanti di altri.

Per tenere in considerazione questa cosa, è possibile assegnare ad ogni processo nel sistema una **priorità** (secondo certe politiche) ed estrarre dalla coda dei processi pronti, per assegnarli la CPU, quello con priorità maggiore.

Il principale problema che può sorgere con questo tipo di schedulazione è la starvation: processi con priorità bassa possono rimanere indefinitamente in attesa che venga loro assegnata la CPU, perché continuano ad "arrivare" nel sistema processi a più alta priorità.

Un possibile modo per risolvere questo problema è la cosiddetta **tecnica dell'aging** (invecchiamento), che consiste nell'andare ad aumentare gradualmente la priorità dei processi che rimangono nello stato di pronto (ready) e, al contrario, diminuire quella dei processi che vengono mandati in esecuzione; così facendo, anche i processi a priorità bassa riusciranno ad ottenere la CPU se rimangono tanto tempo senza essere eseguiti.

## MLFQ (MultiLevel Feedback Queue)

L'algoritmo MLFQ è una strategia avanzata per la schedulazione a breve termine dei processi, in grado di gestirli dinamicamente e adattarsi al comportamento nel tempo di ciascun processo.

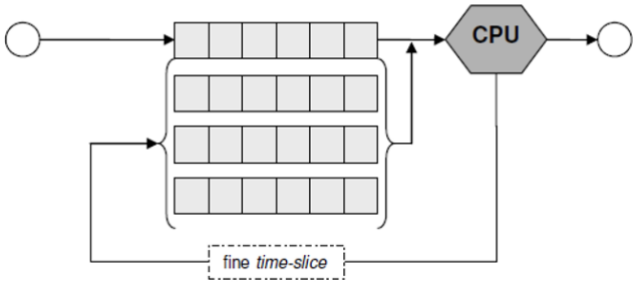
Questo algoritmo prevede non più una singola coda dei processi pronti ad essere eseguiti, ma più code. Queste code differiscono per priorità e time slice; in particolare, la prima coda ha priorità massima e time slice minimo, mentre l'ultima coda ha priorità minima e time slice massimo. Quindi se abbiamo N code, proseguendo dalla coda 1 fino alla coda N la priorità diminuisce, mentre aumenta il time slice assegnato ai processi.

I processi che si trovano in una determinata coda verranno eseguiti solamente quando quelle con priorità maggiore sono tutte vuote.

Le code a priorità più alta sono ideali per i processi interattivi, che non richiedono molto tempo per eseguire, ma che hanno spesso bisogno della CPU, mentre le code con priorità più bassa sono perfette per processi lunghi.

Tipicamente, i processi presenti nell'ultima coda (priorità minore) vengono serviti secondo la logica FCFS.

Quando un nuovo processo viene caricato in memoria RAM, viene inserito nella coda con priorità massima. Se un processo utilizza tutto il time slice che gli viene messo a disposizione, viene considerato CPU-bound (che ricordiamo essere un processo che ha bisogno intensamente del processore) e, quindi, spostato in una coda a priorità inferiore. Se, invece, il processo rilascia spontaneamente la CPU prima del termine del suo quanto di tempo, viene lasciato nella coda attuale. Infine, un processo che ritorna nello stato pronto dopo aver atteso il completamento di una operazione di I/O, viene inserito nella coda a priorità massima.




Come possiamo intuire, questo algoritmo è decisamente più complesso rispetto a quelli analizzati fino ad ora, ma consente di gestire meglio i processi, adattandosi di volta in volta al loro comportamento e migliorando l'utilizzo della CPU rispetto ad un RR puro.

Inoltre, occorre prestare molta attenziona alla scelta dei parametri dell'algoritmo, vale a dire il numero di code, il time slice per ogni coda ed i criteri con cui un processo viene promosso in una coda a priorità maggiore o degradato in una a priorità minore.

Un altro problema che potrebbe presentarsi è sempre la starvation, se continuano ad essere presenti processi nelle code ad alta priorità. Anche qui è possibile mitigare questo inconveniente attraverso una tecnica di aging, consentendo ad un processo presente in una coda a più bassa priorità di essere promosso in una a maggiore priorità se rimane troppo a lungo senza che gli venga assegnata la CPU.